

Lecture 4

Neural Networks: Theory, Architecture, Training

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Agosto 2026

Overview

1. The Multilayer Perceptron
2. Backpropagation
3. Training in Practice
4. Neural Networks as Geometric Ansätze
5. Summary

Recap (Lecture 3): Linear models solved exactly; gradient descent and SGD; regularisation; non-linearity via hand-designed feature maps.

Guiding question: *What happens when the feature map itself is learned?*

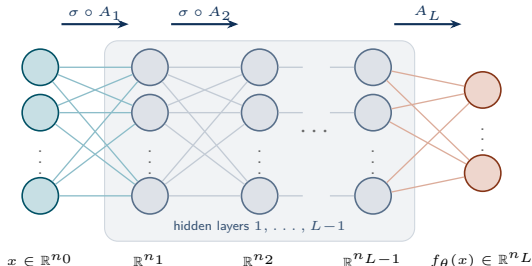
Definition: Compositions of Simple Maps

Multilayer perceptron (MLP)

NNs: A highly-parameterised non-linear map, layering affine maps and non-linear activations. Choose widths $(n_0, n_1, \dots, n_L)$ and an activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$. An MLP is:

$$f_\theta = A_L \circ \sigma \circ A_{L-1} \circ \dots \circ \sigma \circ A_1, \quad A_k(x) = W_k x + b_k, \quad W_k \in \mathbb{R}^{n_k \times n_{k-1}},$$

with depth L ; hidden layers $1, \dots, L-1$; and parameters $\theta = (W_k, b_k)_{k=1}^L$.



- Nodes = coordinates of intermediate spaces $\mathbb{R}^{n_k}$; edges = entries of W_k .
- Comparing Lecture 3: A linear model plus a feature map — the hidden layers are the learned feature map ϕ , now optimised jointly with the final linear layer.

Nonlinearity: Adding Expressivity

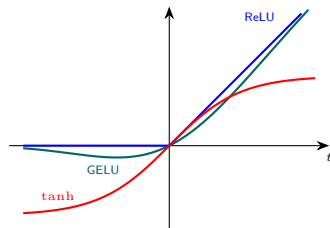
Without σ , depth collapses

- A composition of affine maps is affine: $A_2 \circ A_1 = A'$.
- All expressive power beyond Lecture 3 comes from the nonlinearity (and layering of it).

Common choices for activation σ :

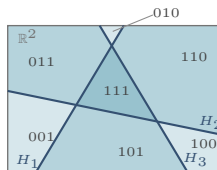
- $\text{ReLU}(t) = \max(0, t)$: default; cheap; no saturation for $t > 0$.
- GELU, SiLU: smooth ReLU variants, standard in transformers.
- $\tanh(t)$: smooth, bounded $\in [-1, 1]$.

...smooth choices good for PINNs (Lecture 12), where network differential properties important.



ReLU NNs = piecewise linear

- f_θ is affine on each cell of a polyhedral complex in input space, a combinatorial-geometric object.
- Expressivity $\approx$ number of linear regions, growing exponentially with depth.



Darker = more neurons on. Each cell s carries its own affine map: $f_\theta(x) = M_s x + c_s$.

Universal Approximation

Theorem (Width-Limit) [Cybenko 1989; Hornik 1991]

For any non-polynomial σ , the family of one-hidden-layer networks is dense in $C(K)$ (uniform norm) for compact $K \subset \mathbb{R}^n$: MLPs can approximate any continuous function arbitrarily well.

Proof idea (Cybenko): if the span were not dense, Hahn–Banach would give a signed measure $\mu \neq 0$ annihilating every $\sigma(w^\top x + b)$; half-space limit arguments force $\mu = 0$ — contradiction.

Theorem (Depth-Limit) [Hanin–Sellke 2017; Lu et al. 2017]

ReLU networks of *fixed width* $n+1$ and arbitrary depth are dense in $C(K)$, while width n is not.

Proof idea: n neurons per layer copy x forward (registers); one extra neuron accumulates a piecewise-linear approximant of f block by block — depth substitutes for width.

What they do **not** say:

- **No rates**: the required width may be astronomical (can grow exponentially in n for fixed accuracy — curse of dimensionality again).
- **No algorithm**: density says a good θ *exists*, not that SGD finds it.
- **No generalisation**: approximating on samples $\neq$ approximating μ -a.e.
- **Depth separations**: some functions need exponentially wide shallow nets but only polynomial deep ones — composition is an efficient prior for hierarchical structure.
- Approximation rates on *manifolds*: for data on $\mathcal{M}^d \subset \mathbb{R}^n$, rates really depend on intrinsic d , not ambient n — geometry cures the curse (cf. Lecture 2).

Backpropagation = Chain Rule on a Graph

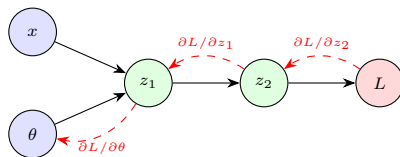
Automatic Differentiation

Record the computation as a DAG; traverse it *backwards*, multiplying Jacobians from the loss:

$$\frac{\partial L}{\partial \theta_k} = \frac{\partial L}{\partial z_L} \frac{\partial z_L}{\partial z_{L-1}} \dots \frac{\partial z_{k+1}}{\partial z_k} \frac{\partial z_k}{\partial \theta_k}.$$

- Exact derivatives (machine precision) — *no* finite differences, *no* symbolic blow-up.
- Cost: one backward pass ($\approx 2\text{--}3\times$ one forward pass), *independent of the number of parameters* $\longrightarrow$ why 10^9 -parameter models are trainable.
- Same machinery differentiates networks w.r.t. *inputs* — the engine of PINNs (Lecture 12).
- For many more outputs than inputs, *forward* mode autodiff is preferable, and uses *dual numbers*:

$$a + b\varepsilon, \quad \varepsilon^2 = 0.$$



Forward pass (black) stores values.
Backward pass (red) propagates gradients.

In PyTorch this is `loss.backward()`:
The graph is built dynamically as your code runs, then differentiated in reverse.

Making Training Work

Training is an art with a checklist

Modern defaults exist for a reason; deviate knowingly.

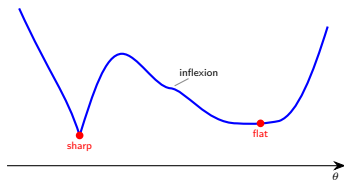
- **Initialisation:** Random weights with variance scaled to preserve signal magnitude layer-to-layer (Xavier/He $\sim 1/n_{k-1}$). Too big $\Rightarrow$ explosion; too small $\Rightarrow$ vanishing gradients.
- **Normalisation layers** (BatchNorm/LayerNorm): Re-centre and re-scale activations; dramatically smooth the effective loss landscape and permit larger learning rates.
- **Optimisers:** *Adam* rescales each coordinate by a running estimate of gradient variance — a diagonal preconditioner, with momentum. Default: Adam with $\eta \approx 10^{-3}$, then tune.
- **Schedules:** warmup then cosine/step decay; small final η polishes the minimum.
- **Monitor:** Observe the training *and* validation curves per epoch — divergence between them is overfitting appearing in real time.

Debugging heuristic

First overfit a tiny subset (loss $\rightarrow 0$ proves the pipeline works); only then train at scale.

The Loss Landscape

- **Nonconvexity:** neural-network losses are complex; in stylized high-dimensional random models, critical points proliferate exponentially and most high-loss critical points are saddles (Hessian has mixed signature generically).
- **Symmetries:** permuting hidden neurons and rescalings (for ReLU) leave f_θ invariant $\Rightarrow$ minima come in high-dimensional orbits/valleys. The landscape is naturally a quotient space.
- **Flat vs sharp:** SGD noise biases towards flat minima; empirically flatter $\Rightarrow$ better generalisation.
- **Overparameterisation** helps optimisation:
 - (i) more params $\Rightarrow$ valleys of zero training loss are connected, so easier to avoid barriers.
 - (ii) init ~ 0 and more params $\Rightarrow$ more low-norm solutions, and lower norm is smoother hence better generalisation, this is *double descent* as test error falls again beyond the interpolation threshold ($|\{\theta_p\}| \sim N$).



One-dimensional representation of $L(\theta)$: sharp and flat minima, connected low-loss regions; the inflexion stands in for a saddle.

Characterising these landscapes:

- Geometry of minima manifolds
- Morse theory of L

...is an active research interface between Geometry and AI.

Regularising Neural Networks

- **Weight decay:** the ℓ^1/ℓ^2 penalties of Lecture 3, applied to all weights; shrinks towards simpler functions.
- **Dropout:** randomly zero a fraction p of activations during training; stabilises and prevents co-adaptation, approximates an ensemble of subnetworks.
- **Early stopping:** halt when *validation* loss stops improving.
- **Data augmentation:** enlarge the dataset by label-preserving transformations (rotations, reflections, reparameterisations). For geometric data: act by the symmetry group of the problem — a poor man's equivariance (the rich man's version is Lecture 10).
- **Get more data:** the most reliable regulariser; in mathematics we can often *generate* data cheaply on demand and at scale — an highly unusual luxury!

Good Practises

For small scientific datasets:

- small network + weight decay + early stopping + heavy use of symmetry
- As much data as is feasible.
- Verify on simple data.
- Tune one thing at a time, on validation data.
- Average over random seeds.

Case Study: Learning Geometric Quantities

Networks as trial functions

An MLP $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a smooth (or piecewise-linear) parametric family — usable exactly where you would use a fourier or finite-element ansatz, but *mesh-free* and *adaptive*.

- **Learning curvature:** train on (point sample, curvature) pairs from surfaces with known geometry; test on unseen surfaces — Tutorial 4's task.
- **Calabi–Yau metrics:** no explicit Ricci-flat metric on a compact CY is known; neural ansätze trained on the Monge–Ampère loss now give the best numerical metrics (Donaldson-era algorithms → modern ML pipelines). A genuine research frontier where this course's toolkit is state of the art.
- **Approximating slow computations:** replace an expensive exact invariant (e.g. from a Gröbner basis computation) by a cheap learned surrogate to *triage* a large search space, then verify candidates exactly.

A common AI workflow in research

Exact mathematics → generate data → train surrogate → explore at scale → **verify exactly**.

Note: NP-type problems have much easier verification, meaning intelligent guesses from AI can find conjecture counter-examples, or highlight mathematical structure that only appears at scale.

Coding Patterns

Example (An MLP in PyTorch (the loop from Lecture 3))

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 64), nn.ReLU(),      # e.g. input is 3d coords of a point
    nn.Linear(64, 64), nn.ReLU(),
    nn.Linear(64, 1)                 # e.g. output is curvature at a point
)

opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- `nn.Module` composes: build big models from small, tested pieces.
- Shapes first: annotate the tensor shape at every stage; most bugs are shape bugs.
- Hygiene: fixed seeds, config files, one Git commit per experiment, save test data and loss curves — replotting doesn't need rerunning.
- Let your AI assistant make a first draft; *you* specify architecture, loss, and testing protocol.

AI Agents in the Development Loop

- Specify what you want: data representation, architecture, loss, testing protocol.
- Prompt your AI assistant to build the first draft (its likely seen equivalent code many times).
- Review the code outputs and format.
- Iterate agentic edits and testing.

Summary

Key points

- MLPs are compositions of affine maps and nonlinear activations (alike linear models with a *learned* feature map).
- Universal approximation guarantees density, not {efficiency, trainability, generalisation} — but at least the function class is not the limiting factor.
- Backpropagation is the chain rule run backwards over a computation graph, clever tricks ensure compute cost comparable to inference.
- Training practice: scale-aware initialisation, layer normalisation, adaptive optimiser (Adam), learning rate schedules, and validation-driven early stopping.
- The loss landscape is a rich geometric object: saddles, symmetry orbits of minima, enormously varying curvature — and an active open research area.
- Neural networks are *mesh-free* ansatz spaces — learn {curvatures, metrics, forms, etc.} *smoothly* over the *full* data manifold.

Next

Tutorial 4: MLPs approximating geometric quantities; vs polynomial baselines.

Lecture 5: Convolutions — symmetry enters the architecture (CNNs).